2447116_CIA3_2

December 1, 2025

[5]:
```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,␣
 ↪Dropout, BatchNormalization, Input
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.optimizers import Adam
from sklearn.metrics import classification_report, confusion_matrix
import numpy as np
import matplotlib.pyplot as plt
import os

tf.random.set_seed(42)
np.random.seed(42)
```

0.1 1. Data Preprocessing

[6]:
```
base_dir = 'chest_xray-20251201T045557Z-1-001/chest_xray'
train_dir = os.path.join(base_dir, 'train')
test_dir = os.path.join(base_dir, 'test')
val_dir = os.path.join(base_dir, 'val')

IMG_HEIGHT = 150
IMG_WIDTH = 150
BATCH_SIZE = 32

train_datagen = ImageDataGenerator(
    rescale=1./255,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True
)

# Rescaling for Validation and Test
val_test_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    train_dir,
```

```
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary'
)

validation_generator = val_test_datagen.flow_from_directory(
    val_dir,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary'
)

test_generator = val_test_datagen.flow_from_directory(
    test_dir,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    shuffle=False
)
```

```
Found 5216 images belonging to 2 classes.
Found 16 images belonging to 2 classes.
Found 624 images belonging to 2 classes.
```

0.2 2. Build CNN Model

[7]:
```
def build_model():
    model = Sequential([
        Input(shape=(IMG_HEIGHT, IMG_WIDTH, 3)),

        Conv2D(32, (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        MaxPooling2D((2, 2)),
        Dropout(0.2),

        Conv2D(64, (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        MaxPooling2D((2, 2)),
        Dropout(0.2),

        Conv2D(128, (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        MaxPooling2D((2, 2)),
        Dropout(0.2),

        Conv2D(256, (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        MaxPooling2D((2, 2)),
```

```
        Dropout(0.2),

        # Dense Layers
        Flatten(),
        Dense(512, activation='relu'),
        Dropout(0.5),
        Dense(1, activation='sigmoid')
    ])
    return model

model = build_model()
model.summary()
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 150, 150, 32)	896
batch_normalization_4 (BatchNormalization)	(None, 150, 150, 32)	128
max_pooling2d_4 (MaxPooling2D)	(None, 75, 75, 32)	0
dropout_5 (Dropout)	(None, 75, 75, 32)	0
conv2d_5 (Conv2D)	(None, 75, 75, 64)	18,496
batch_normalization_5 (BatchNormalization)	(None, 75, 75, 64)	256
max_pooling2d_5 (MaxPooling2D)	(None, 37, 37, 64)	0
dropout_6 (Dropout)	(None, 37, 37, 64)	0
conv2d_6 (Conv2D)	(None, 37, 37, 128)	73,856
batch_normalization_6 (BatchNormalization)	(None, 37, 37, 128)	512
max_pooling2d_6 (MaxPooling2D)	(None, 18, 18, 128)	0
dropout_7 (Dropout)	(None, 18, 18, 128)	0
conv2d_7 (Conv2D)	(None, 18, 18, 256)	295,168

```
batch_normalization_7            (None, 18, 18, 256)             1,024
(BatchNormalization)

max_pooling2d_7 (MaxPooling2D)   (None, 9, 9, 256)                0

dropout_8 (Dropout)              (None, 9, 9, 256)               0

flatten_1 (Flatten)              (None, 20736)                   0

dense_2 (Dense)                  (None, 512)                10,617,344

dropout_9 (Dropout)              (None, 512)                     0

dense_3 (Dense)                  (None, 1)                     513
```

Total params: 11,008,193 (41.99 MB)

Trainable params: 11,007,233 (41.99 MB)

Non-trainable params: 960 (3.75 KB)

0.3 3. Training with Weighted Binary Cross-Entropy

[8]:
```
num_normal = len(os.listdir(os.path.join(train_dir, 'NORMAL')))
num_pneumonia = len(os.listdir(os.path.join(train_dir, 'PNEUMONIA')))
total = num_normal + num_pneumonia

weight_for_0 = (1 / num_normal) * (total / 2.0)
weight_for_1 = (1 / num_pneumonia) * (total / 2.0)

class_weight = {0: weight_for_0, 1: weight_for_1}

print(f"Weight for Normal (0): {weight_for_0:.2f}")
print(f"Weight for Pneumonia (1): {weight_for_1:.2f}")

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy', tf.keras.metrics.Precision(name='precision'), tf.keras.
 ↪metrics.Recall(name='recall')]
)

history = model.fit(
    train_generator,
```

```
    steps_per_epoch=train_generator.samples // BATCH_SIZE,
    epochs=5,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // BATCH_SIZE,
    class_weight=class_weight
)
```

```
Weight for Normal (0): 1.94
Weight for Pneumonia (1): 0.67
Epoch 1/5
163/163             203s 1s/step -
accuracy: 0.8344 - loss: 1.5089 - precision: 0.9373 - recall: 0.8328 -
val_accuracy: 0.5000 - val_loss: 39.0234 - val_precision: 0.5000 - val_recall:
1.0000
Epoch 2/5
163/163             192s 1s/step -
accuracy: 0.9015 - loss: 0.2759 - precision: 0.9706 - recall: 0.8945 -
val_accuracy: 0.5000 - val_loss: 28.8223 - val_precision: 0.5000 - val_recall:
1.0000
Epoch 3/5
163/163             192s 1s/step -
accuracy: 0.9126 - loss: 0.2351 - precision: 0.9758 - recall: 0.9048 -
val_accuracy: 0.6250 - val_loss: 2.7797 - val_precision: 0.5714 - val_recall:
1.0000
Epoch 4/5
163/163             198s 1s/step -
accuracy: 0.9275 - loss: 0.2038 - precision: 0.9794 - recall: 0.9218 -
val_accuracy: 0.5000 - val_loss: 21.8209 - val_precision: 0.5000 - val_recall:
1.0000
Epoch 5/5
163/163             188s 1s/step -
accuracy: 0.9294 - loss: 0.1863 - precision: 0.9811 - recall: 0.9228 -
val_accuracy: 0.5000 - val_loss: 4.4686 - val_precision: 0.0000e+00 -
val_recall: 0.0000e+00
```

0.4 4. Evaluation

[9]:
```
test_loss, test_acc, test_precision, test_recall = model.
  ↪evaluate(test_generator)

f1_score = 2 * (test_precision * test_recall) / (test_precision + test_recall)

print(f"\nTest Accuracy: {test_acc:.4f}")
print(f"Test Precision: {test_precision:.4f}")
print(f"Test Recall: {test_recall:.4f}")
print(f"Test F1 Score: {f1_score:.4f}")
```

```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs_range = range(len(acc))

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```

```
20/20              7s 331ms/step -
accuracy: 0.5064 - loss: 7.5113 - precision: 0.9362 - recall: 0.2256

Test Accuracy: 0.5064
Test Precision: 0.9362
Test Recall: 0.2256
Test F1 Score: 0.3636
```